Profiling SQL Queries

Table Of Contents

1. Overview

SQLite contains built-in support for profiling SQL queries, but it is not enabled by default. In order to enable support for query profiling, SQLite must be compiled with the following option:

```
-DSQLITE_ENABLE_STMT_SCANSTATUS
```

Building SQLite with this option enables the sqlite3_stmt_scanstatus_v2() API, which provides access to the various profiling metrics. The remainder of this page discusses the profiling reports generated by the SQLite command-line shell using these metrics, rather than the API directly.

The profiling reports generated by the shell are very similar to the query plan reports generated by the EXPLAIN QUERY PLAN command. This page assumes that the reader is familiar with this format.

Within a command-line shell compiled with the option above, query profiling is enabled using the ".scanstats on" command:

```
sqlite> .scanstats on
```

Once enabled, the shell automatically outputs a query profile after each SQL query executed. Query profiling can be disabled using ".scanstats off". For example:

```
sqlite> .scanstats on
sqlite> SELECT a FROM t1, t2 WHERE a IN (1,2,3) AND a=d+e;
QUERY PLAN (cycles=255831538 [100%])
|--SEARCH t1 USING INTEGER PRIMARY KEY (rowid=?)     (cycles=60048488 [23%] loops=1 rows=3)
`--SCAN t2                                           (cycles=133558052 [52%] loops=3 rows=150000)
```

2. Simple Cases - Rows, Loops and Cycles

Consider a database with the following schema:

```
CREATE VIRTUAL TABLE ft USING fts5(text);
CREATE TABLE t1(a, b);
CREATE TABLE t2(c INTEGER PRIMARY KEY, d);
```

Then, after first executing ".scanstats on":

```
sqlite3> SELECT * FROM t1, t2 WHERE t2.c=t1.a;
<...query results...>
QUERY PLAN (cycles=1140768 [100%])
|--SCAN t1                                           (cycles=455974 [40%] loops=1 rows=500)
`--SEARCH t2 USING INTEGER PRIMARY KEY (rowid=?)     (cycles=619820 [54%] loops=500 rows=250)
```

The text in the example above following the snipped "<...query results...>" is the profile report for the join query just executed. The parts of the profile report that are similar to the EXPLAIN QUERY PLAN output indicate that the query is implemented by doing a full table-scan of table "t1", with a lookup by INTEGER PRIMARY KEY on table "t2" for each row visited.

The "loops=1" notation on the "SCAN t1" line indicates that this loop - the full-table scan of table "t1" - ran exactly once. "rows=500" indicates that that single scan visited 500 rows.

The "SEARCH t2 USING ..." line contains the annotation "loops=500" to indicate that this "loop" (really a lookup by INTEGER PRIMARY KEY) ran 500 times. Which makes sense - it ran once for each row visited by the full-table scan of "t1". "rows=250" means that,

altogether, those 500 loops visited 250 rows. In other words, only half of the INTEGER PRIMARY KEY lookups on table t2 were successful, for the other half of the lookups there was no row to find.

The loop-count for a SEARCH or SCAN entry is not necessarily the same as the number of rows output by the outer loop. For example, if the query above were modified as follows:

```
sqlite3> SELECT * FROM t1, t2 WHERE t1.b<=100 AND t2.c=t1.a;
<...query results...>
QUERY PLAN (cycles=561002 [100%])
|--SCAN t1                                          (cycles=345950 [62%] loops=1 rows=500)
`--SEARCH t2 USING INTEGER PRIMARY KEY (rowid=?)    (cycles=128690 [23%] loops=100 rows=50)
```

This time, even though the "SCAN t1" loop still visits 500 rows, the "SEARCH t2" lookup is only performed 100 times. This is because SQLite was able to discard rows from t1 that did not match the "t1.b<=100" constraint.

The "cycles" measurements are based on the CPU time-stamp counter , and so are a good proxy for wall-clock time. For the query above, the total number of cycles was 561002. For each of the two loops ("SCAN t1..." and "SEARCH t2..."), the cycles count represents the time spent in operations that can be directly attributed to that loop only. Specifically, this is the time spent navigating and extracting data from the database b-trees for that loop. These values never quite add up to the total cycles for the query, as there are other internal operations performed by SQLite that are not directly attributable to either loop.

The cycles count for the "SCAN t1" loop was 345950 - 62% of the total for the query. The 100 lookups peformed by the "SEARCH t1" loop took 128690 cycles, 23% of the total.

When a virtual table is used, the "rows" and "loops" metrics have the same meaning as for loops on regular SQLite tables. The "cycles" meaurement is the total cycles consumed within virtual table methods associated with the loop. For example:

```
sqlite3> SELECT * FROM ft('sqlite'), t2 WHERE t2.c=ft.rowid;
<...query results...>
QUERY PLAN (cycles=836434 [100%]
|--SCAN ft VIRTUAL TABLE INDEX 0:M1                 (cycles=739602 [88%] loops=1 rows=48)
`--SEARCH t2 USING INTEGER PRIMARY KEY (rowid=?)    (cycles=62866 [8%] loops=48 rows=25)
```

In this case the single query (loops=1) on fts5 table "ft" returned 48 rows (rows=48) and consumed 739602 cycles (cycles=739602), which was roughly 88% of the total query time.

3. Complex Cases - Rows, Loops and Cycles

Using the same schema as in the previous section, consider this more complicated example:

```
sqlite3> WITH cnt(i) AS (
  SELECT 1 UNION SELECT i+1 FROM cnt WHERE i<100
)
SELECT
  *, (SELECT d FROM t2 WHERE c=ft.rowid)
FROM
  (SELECT count(*), a FROM t1 GROUP BY a) AS v1 CROSS JOIN
  ft('sqlite'),
  cnt
WHERE cnt.i=ft.rowid AND v1.a=ft.rowid;
<...query results...>
QUERY PLAN (cycles=177665334 [100%])
|--CO-ROUTINE v1                                    (cycles=4500444 [3%])
|  |--SCAN t1                                       (cycles=397052 [0%] loops=1 rows=500)
|  `--USE TEMP B-TREE FOR GROUP BY
|--MATERIALIZE cnt                                  (cycles=1275068 [1%])
|  |--SETUP
|  |  `--SCAN CONSTANT ROW
|  `--RECURSIVE STEP
|     `--SCAN cnt                                   (cycles=129166 [0%] loops=100 rows=100)
|--SCAN v1                                          (loops=1 rows=500)
|--SCAN ft VIRTUAL TABLE INDEX 0:M1=                (cycles=161874340 [91%] loops=500 rows=271)
|--SCAN cnt                                         (cycles=7336350 [4%] loops=95 rows=9500)
`--CORRELATED SCALAR SUBQUERY 3                     (cycles=168538 [0%] loops=37)
   `--SEARCH t2 USING INTEGER PRIMARY KEY (rowid=?) (cycles=94724 [0%] loops=37 rows=21)
```

The most complicated part of the example above is understanding the query plan - the portion of the report that would also be generated by an EXPLAIN QUERY PLAN command. Other points of interest are:

- Sub-query "v1" is implemented as a co-routine. In this case the sub-query is reported on separately, and a "cycles" count is available for the entire sub-query. There is also a "SCAN v1" line - this represents the invocation of the sub-query co-routine from the main query. This entry has no cycles associated with it, as the entire cost of the sub-query is attributed to the co-routine. It does have "loops" and "rows" values - the sub-query is scanned once and returns 500 rows.

- Recursive sub-query "cnt" is materialized (cached in a temp table) before the main query is run. The entire cost of the materialization is attributed to the "MATERIALIZE cnt" element. There is also a "SCAN cnt" item representing the scans of the materialized sub-query. The cycles value associated with this item represents the time taken to scan the temp-table containing the materialized sub-query, which is separate from the cycles used to populate it.

- There are cycles and loops measurements for the scalar sub-query as well. These represent the total cycles consumed while executing the sub-query and the number of times it was executed, respectively.

- Where one item is a parent of another, as in "CORRELATED SCALAR SUBQUERY 3" and "SEARCH t2 USING...", then the cycles value associated with the parent includes those cycles associated with all child elements. In all cases, the percentage values relate to the total cycles used by the query, not the cycles used by the parent.

The following query uses an automatic index and an external sort:

```
sqlite> SELECT * FROM
  t2,
  (SELECT count(*) AS cnt, d FROM t2 GROUP BY d) AS v2
WHERE v2.d=t2.d AND t2.d>100
ORDER BY v2.cnt;
<...query results...>
QUERY PLAN (cycles=6234376 [100%])
|--MATERIALIZE v2                                    (cycles=2351916 [38%])
|  |--SCAN t2                                        (cycles=188428 [3%] loops=1 rows=250)
|  `--USE TEMP B-TREE FOR GROUP BY
|--SCAN t2                                           (cycles=455736 [7%] loops=1 rows=250)
|--CREATE AUTOMATIC INDEX ON v2(d, cnt)              (cycles=1668380 [27%] loops=1 rows=250)
|--SEARCH v2 USING AUTOMATIC COVERING INDEX (d=?)    (cycles=932824 [15%] loops=200 rows=200)
`--USE TEMP B-TREE FOR ORDER BY                      (cycles=662456 [11%] loops=1 rows=200)
```

Points of interest are:

- This query materializes the sub-query into a temp table, then creates an automatic (i.e. transient) index on it, then uses that index to optimize the join. All three of these steps - "MATERIALIZE v2", "CREATE AUTOMATIC INDEX" and "SEARCH ... USING AUTOMATIC INDEX" have separate cycle counts. The "rows" associated with the "CREATE AUTOMATIC INDEX" line represents the total number of rows included in the index. The "loops" and "rows" associated with the "SEARCH ... USING AUTOMATIC INDEX" line represent the number of lookups the index was used for and the total number of rows found by those lookups.

- The external sort "USE TEMP B-TREE FOR ORDER BY" is also accounted for separately. The cycles count represents the extra cycles consumed by sorting the returned rows, above those that would have been used if the rows were returned in arbitrary order. The rows count represents the number of rows sorted.

4. Planner Estimates

As well as ".scanstats on" to enable profiling and ".scanstats off" to disable it, the shell also accepts ".scanstats est":

```
sqlite> .scanstats est
```

This enables a special kind of profiling report that includes two extra values associated with each "SCAN..." and "SEARCH..." element of a query profile:

- **rpl** (rows per loop): This is the value of the "rows" metric divided by that of the "loops" metric.
- **est** (estimate): This is the number of rows per loop that the query planner estimated would be returned by the current query element. If this value is radically different from the actual rows per loop value and query performance is unsatisfactory, the quality of this estimate may be improved by running ANALYZE.

```
sqlite> SELECT a FROM t1, t2 WHERE a IN (1,2,3) AND a=d+e ORDER BY a;
<query results...>
QUERY PLAN (cycles=264725190 [100%]
|--SEARCH t1 USING INTEGER PRIMARY KEY (rowid=?)     (cycles=60511568 [23%] loops=1 rows=3 rpl=3.0 est=3.0)
`--SCAN t2                                           (cycles=139461608 [53%] loops=3 rows=150000 rpl=50000.0 est=1048576.0)
```

5. Low-Level Profiling Data

Also supported is the ".scanstats vm" command. This enables a lower-level profiling report showing the number of times each VM instruction was executed and the percentage of clock-cycles that passed while it was executing:

```
sqlite> .scanstats vm
```

Then:

```
sqlite> SELECT count(*) FROM t2 WHERE (d % 5) = 0;
<query results...>
addr  cycles  nexec   opcode         p1    p2    p3    p4             p5  comment
----  ------  ------  -------------  ----  ----  ----  -------------  --  -------------
0     0.0%    1       Init           1     18    0                    0   Start at 18
1     0.0%    1       Null           0     1     1                    0   r[1..1]=NULL
2     0.0%    1       OpenRead       0     2     0     2              0   root=2 iDb=0; t2
3     0.0%    1       ColumnsUsed    0     0     0     2              0
4     0.0%    1       Explain        4     0     0     SCAN t2        0
5     0.0%    1       CursorHint     0     0     0     EQ(REM(c1,5),0) 0
6     0.0%    1       Rewind         0     14    0                    0
7     46.86%  150000    Column         0     1     3                    0   r[3]= cursor 0 column 1
8     18.94%  150000    Remainder      4     3     2                    0   r[2]=r[3]%r[4]
9     5.41%   150000    ReleaseReg     3     1     0                    0   release r[3] mask 0
10    12.09%  150000    Ne             5     13    2                    80  if r[2]!=r[5] goto 13
11    1.02%   30000     ReleaseReg     2     1     0                    0   release r[2] mask 0
12    2.95%   30000     AggStep1       0     0     1     count(0)       0   accum=r[1] step(r[0])
13    12.72%  150000  Next           0     7     0                    1
14    0.0%    1       AggFinal       1     0     0     count(0)       0   accum=r[1] N=0
15    0.0%    1       Copy           1     6     0                    0   r[6]=r[1]
16    0.0%    1       ResultRow      6     1     0                    0   output=r[6]
17    0.01%   1       Halt           0     0     0                    0
18    0.0%    1       Transaction    0     0     1     0              1   usesStmtJournal=0
19    0.0%    1       Integer        5     4     0                    0   r[4]=5
20    0.0%    1       Integer        0     5     0                    0   r[5]=0
21    0.0%    1       Goto           0     1     0                    0
```